Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory Work #5

---

# Dual-Sensor Temperature Monitor

**Signal Acquisition, Binary Threshold Conditioning, and LCD Reporting**

---

*Author:*
Sava Luchian
std. gr. FAF-233

*Verified:*
Alexei Martiniuc

Chisinau 2026

# Technical Task

## Purpose of the Laboratory

Familiarise students with sensor signal acquisition and conditioning techniques on embedded microcontrollers by implementing a dual-sensor temperature monitoring application that periodically reads two sensors (one analogue NTC thermistor, one digital DHT22), applies a binary threshold filter with hysteresis and persistence confirmation to each channel, drives visual and audible alerts, and reports all intermediate processing states via STDIO (`printf`) and a 16×2 LCD display.

## Objectives

1. Read raw data from an analogue sensor (NTC thermistor via ADC) and a digital sensor (DHT22 via 1-Wire) at a configurable acquisition rate (40 ms).
2. Read a potentiometer on A1 and map it to a configurable temperature setpoint in the range 20–35 °C (bonus functionality).
3. Apply a multi-stage signal conditioning pipeline: voltage conversion, B-model resistance-to-temperature calculation, binary threshold detection with hysteresis and persistence debouncing.
4. Display processed sensor values, conditioning state, and alert status on a 16×2 LCD and via STDIO `printf` at a 500 ms reporting rate.
5. Drive a green/red LED pair and a passive buzzer to provide immediate alert feedback.
6. Demonstrate clean modular software architecture separating sensors, signal conditioning, peripherals, I/O, and application logic.

## Individual Task

Design and implement on an **Arduino Uno (ATmega328P)** a bare-metal (non-FreeRTOS) dual-sensor monitoring application structured as follows:

- **NTC Sensor** – Analogue thermistor on pin A0; read ADC raw, convert to voltage, compute resistance via voltage-divider model, compute temperature with the B-parameter (Steinhart–Hart) equation.
- **DHT22 Sensor** – Digital temperature/humidity sensor on D8; read via Adafruit DHT library; cache last valid value when the sensor returns NaN within the 2 s recharge window.

- **Potentiometer (Bonus)** – Analogue input on A1; mapped to setpoint interval 20–35 °C. Dynamic thresholds are computed as: `high=setpoint+1.0`, `low=setpoint-1.0`.
- **Binary Threshold Filter** – One instance per sensor; hysteresis thresholds are updated at runtime from the potentiometer setpoint; persistence confirmation: 2 consecutive samples before state change ($\leq 80$ ms confirmation at 40 ms acquisition); prevents oscillation near the threshold.
- **Alert Output** – **Red LED** + **Buzzer ON** when either sensor triggers `stableHigh`; **Green LED** otherwise.
- **Reporting** – Every 500 ms: `printf` full diagnostic dump (raw ADC, voltage, temperature, candidateHigh, stableHigh, pending counter) for both channels plus potentiometer raw/setpoint and thresholds; LCD shows `N:xx.x D:xx.x` and `TH:high/low`.

# 1 Domain Analysis

## 1.1 Technological Context and Application Domain

Temperature monitoring is one of the most common applications in embedded systems. From industrial process control to consumer IoT devices, reliable acquisition and conditioning of thermal data underpins safe and correct system operation. This laboratory targets the full signal processing chain from physical transducer to user-visible output:

- **Analogue sensing**: An NTC (Negative Temperature Coefficient) thermistor changes resistance non-linearly with temperature. The MCU's 10-bit ADC converts the divider voltage to a digital value; software then applies the B-parameter (Steinhart–Hart) model to recover temperature in °C.
- **Digital sensing**: The DHT22 (AM2302) encodes temperature and humidity as a single-wire serial bitstream. The Adafruit DHT library handles the protocol timing; the application layer deals only with floating-point temperature values and error recovery.
- **Binary threshold conditioning**: Raw temperature readings are inherently noisy near a decision boundary. A hysteresis band (high/low threshold pair) combined with a persistence counter prevents repeated alert toggling when the temperature hovers around the set-point – a standard industrial technique also known as "bang-bang with hysteresis."

- **Multi-rate loop**: Acquisition runs at 40 ms (sensor-dictated minimum); LCD/STDIO reporting runs at 500 ms to avoid flooding the display. This decoupling of acquisition from reporting is the canonical embedded real-time pattern.

## 1.2 Hardware Components and Justification

| Component | Qty | Role in the application |
|---|---|---|
| Arduino Uno (ATmega328P) | 1 | Main MCU; 16 MHz, 2 KB SRAM, 32 KB Flash. 10-bit ADC for NTC; GPIO for DHT22 and LCD. |
| NTC Thermistor (10 kΩ @ 25 °C, B=3950 K) | 1 | Analogue temperature sensor; connected with a 10 kΩ fixed resistor in voltage-divider configuration to A0. |
| Potentiometer (10 kΩ) | 1 | User-controlled setpoint selector on A1. Mapped in software to 20–35 °C and used to compute dynamic threshold pair. |
| DHT22 (AM2302) | 1 | Digital temperature + humidity sensor; 1-Wire protocol on D8. Accuracy ±0.5 °C. |
| 16×2 LCD (LiquidCrystal, 4-bit) | 1 | Real-time display of NTC temperature, DHT22 temperature, and alert state. |
| Green LED + 220 Ω | 1 | Normal-operation indicator (no alert). |
| Red LED + 220 Ω | 1 | Alert indicator (either sensor above threshold). |
| Passive buzzer | 1 | Audible alert at 1 kHz when alert is active. |
| 10 kΩ resistor (fixed divider) | 1 | Voltage-divider partner for NTC. |
| Breadboard + jumper wires | – | Prototyping interconnection. |
| USB cable | 1 | 5 V power and serial communication (115 200 baud). |

Table 1: Hardware components used in Lab 5

## 1.3 Software Components and Technologies

- **PlatformIO + VS Code**: Single `uno` environment. Arduino framework, no RTOS.
- **Arduino Framework (avr-arduino)**: GPIO, ADC (`analogRead`), `millis()`, `tone`/`noTone`, `LiquidCrystal` library.
- **Adafruit DHT sensor library 1.4.6**: Handles the DHT22 single-wire protocol; internally caches readings for 2 s.
- **arduino-libraries/LiquidCrystal 1.0.7**: 4-bit parallel interface to HD44780-compatible 16×2 LCD.
- **AVR-libc stdio**: `fdev_setup_stream` + `printf` redirected to Serial UART via `StdioBridge`. `dtostrf()` used for float formatting (AVR has no `%f` in default `printf`).
- **Wokwi Simulator**: Circuit and firmware simulation in VS Code for functional testing.

## 1.4 Analysis of Existing Solutions

Several design decisions were made over alternative approaches:

- **Bare-metal vs. FreeRTOS**: Because the acquisition pipeline is simple (two sensors, one conditioning step, one report) and latency requirements are soft ($\leq 100$ ms), a cooperative bare-metal loop using `millis()` timestamps is sufficient and avoids the FreeRTOS RAM overhead ($\approx 400$ B for task stacks + TCBs) on the 2 KB ATmega328P.
- **NTC B-parameter vs. lookup table**: The B-parameter (Steinhart–Hart two-point) model requires only two ROM constants and one `log()` call. A LUT requires 20–30 entries plus interpolation code. For the target 0–50 °C range the B-model error is $< 0.5$ °C – acceptable for this application.
- **Hysteresis + persistence vs. simple threshold**: A single threshold produces rapid oscillation when the temperature fluctuates near the set-point. The hysteresis band around a dynamic setpoint (controlled by potentiometer) prevents state changes in the neutral zone; the 2-sample persistence filter additionally suppresses single-sample noise spikes.
- **DHT22 vs. DHT11**: DHT22 provides ±0.5 °C accuracy vs. ±2 °C for DHT11, and covers a wider range. The 2 s minimum read interval is handled transparently by the Adafruit library.

### 1.5 Real-World Applications

The design patterns in this laboratory appear directly in commercial embedded products:

- **HVAC controllers**: Hysteresis-based thermostat logic (identical to `BinaryThresholdFilter`) prevents compressor short-cycling by requiring temperature to fall below the low threshold before the cooling call is removed.
- **Industrial process monitors**: Dual-sensor redundancy (NTC + DHT22) is standard practice for critical temperature monitoring – if one sensor fails, the other provides fallback.
- **Cold-chain logistics**: NTC thermistor arrays with B-model conversion are standard in IoT temperature loggers for pharmaceutical and food transport.
- **Server room monitoring**: LCD + STDIO reporting mirrors rack monitoring panels that show live sensor data and alarm state simultaneously on a local display and a remote serial/network interface.
- **Fire alarm panels**: Persistence confirmation (requiring stable over-threshold readings for N consecutive samples) is mandatory in EN 54-compliant fire detectors to prevent false alarms from transient heat sources.

## 2 Conceptualisation and Design

### 2.1 Technical Requirements

| ID | Category | Requirement | Type |
|---|---|---|---|
| R-F-01 | NTC | System shall read NTC ADC raw value at every acquisition cycle. | Func. |
| R-F-02 | NTC | System shall convert ADC raw to voltage, resistance, and temperature (B-model). | Func. |
| R-F-03 | DHT22 | System shall read DHT22 temperature at every acquisition cycle; cache last valid value on NaN. | Func. |
| R-F-04 | Conditioning | System shall apply hysteresis threshold per sensor with dynamic pair `high=setpoint+1.0`, `low=setpoint-1.0`. | Func. |

| | | | |
|---|---|---|---|
| R-F-05 | Conditioning | System shall require 2 consecutive confirming samples before changing stable state. | Func. |
| R-F-05a | Potentiometer | System shall read potentiometer (A1) and map ADC 0..1023 to setpoint 20..35 °C. | Func. |
| R-F-06 | Alert | Red LED shall illuminate when either sensor `stableHigh` is true. | Func. |
| R-F-07 | Alert | Green LED shall illuminate when no sensor `stableHigh` is active. | Func. |
| R-F-08 | Alert | Buzzer shall sound at 1 kHz when alert is active; shall be silent otherwise. | Func. |
| R-F-09 | LCD | LCD row 0 shall show NTC temperature and DHT22 temperature (or "n/a" if invalid). | Func. |
| R-F-10 | LCD | LCD row 1 shall show live threshold pair in format `TH:high/low` and alert marker. | Func. |
| R-F-11 | STDIO | Every 500 ms: `printf` NTC raw, voltage, temperature, conditioner state, DHT22 temperature, conditioner state, and system alert. | Func. |
| R-F-12 | STDIO | System shall print configuration banner on startup via `printf`. | Func. |
| R-NF-01 | Latency | Alert state shall reflect sensor readings within 100 ms of threshold crossing. | Non-func. |
| R-NF-02 | Memory | Firmware shall fit within 32 KB Flash and 2 KB SRAM. | Non-func. |
| R-NF-03 | Modularity | Application shall be structured in dedicated modules: config, sensors, signal, peripherals, io, app. | Non-func. |
| R-NF-04 | STDIO | All text output shall use `printf` via a `stdout` → Serial bridge. | Non-func. |
| R-NF-05 | Robustness | System shall not crash when DHT22 returns NaN; cached value shall be used. | Non-func. |

Table 2: Technical requirements for Lab 5

## 2.2 Architectural Design

### 2.2.1 System Structural Architecture

The application follows a five-layer hardware–software boundary:

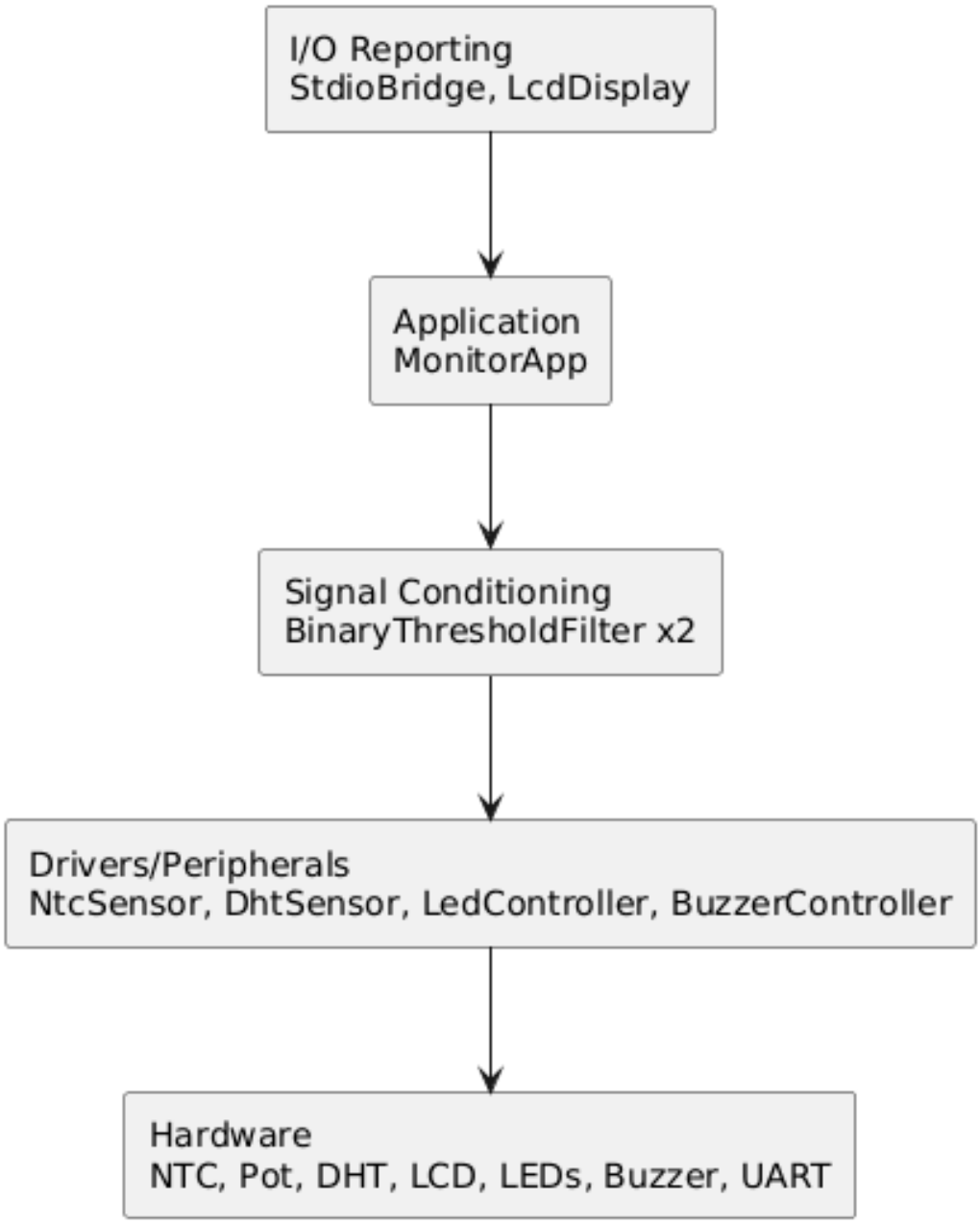



Figure 1: Layered software architecture – Lab 5

### 2.2.2 HW/SW Interface and Component Interaction

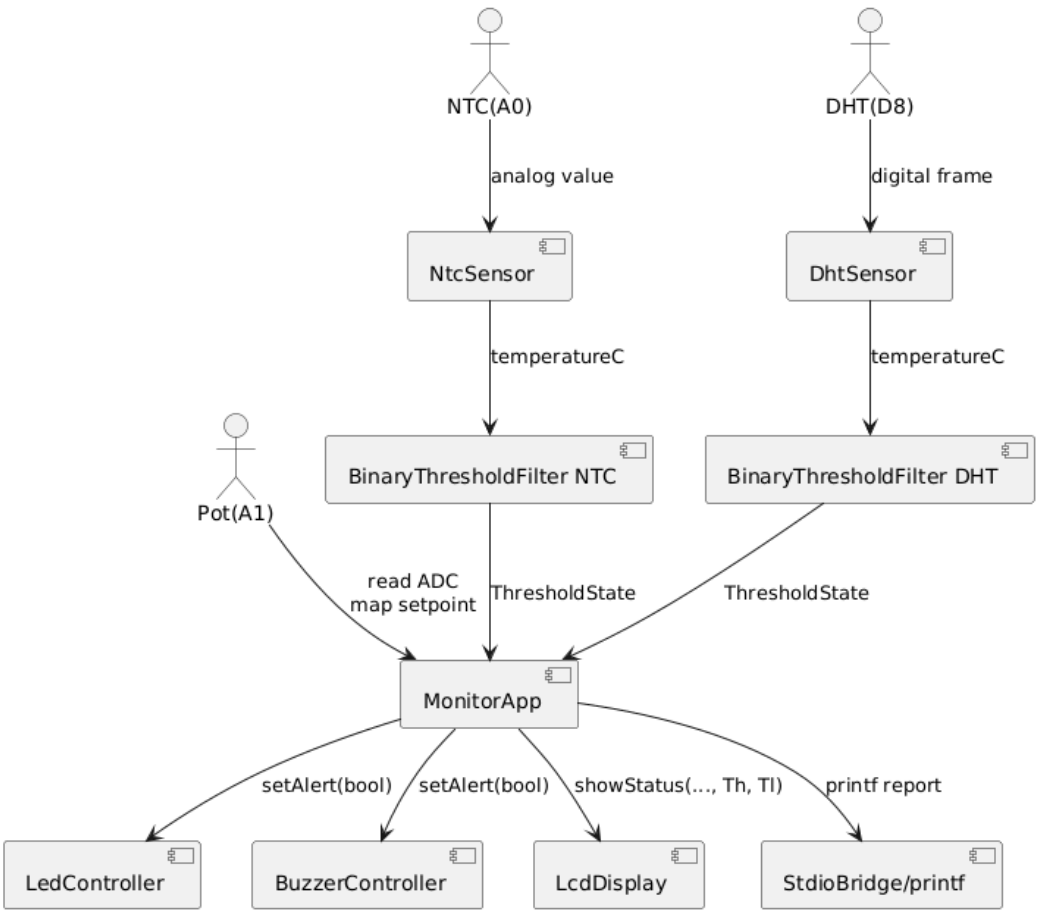
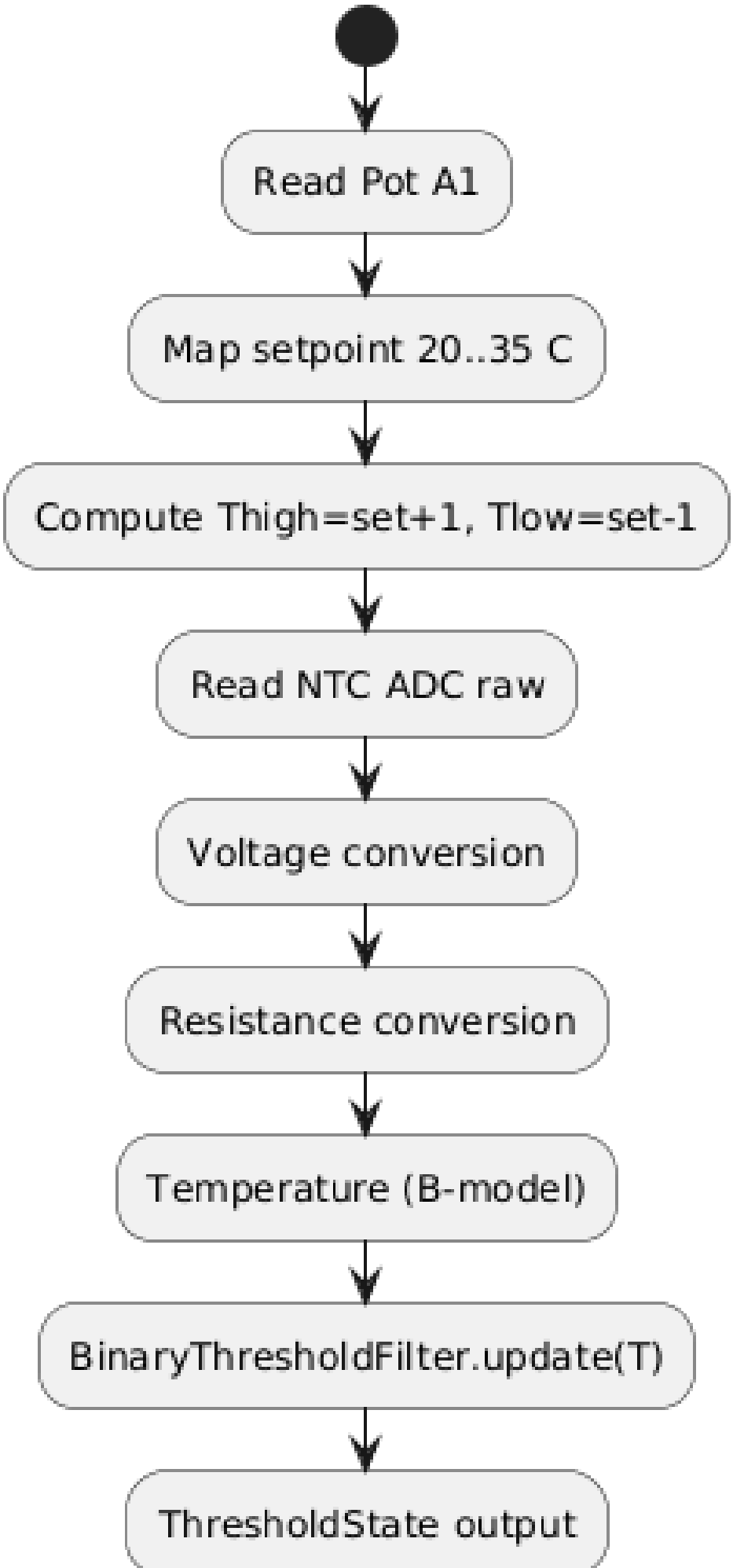



Figure 2: Component interaction diagram – Lab 5

### 2.2.3 Signal Conditioning Pipeline Design

The NTC signal conditioning pipeline (plus dynamic threshold update from potentiometer) consists of sequential stages:


Read Pot A1
Map setpoint 20..35 C
Compute Thigh=set+1, Tlow=set-1
Read NTC ADC raw
Voltage conversion
Resistance conversion
Temperature (B-model)
BinaryThresholdFilter.update(T)
ThresholdState output

### 2.2.4 BinaryThresholdFilter State Machine

The filter implements a two-state machine with hysteresis and persistence confirmation:

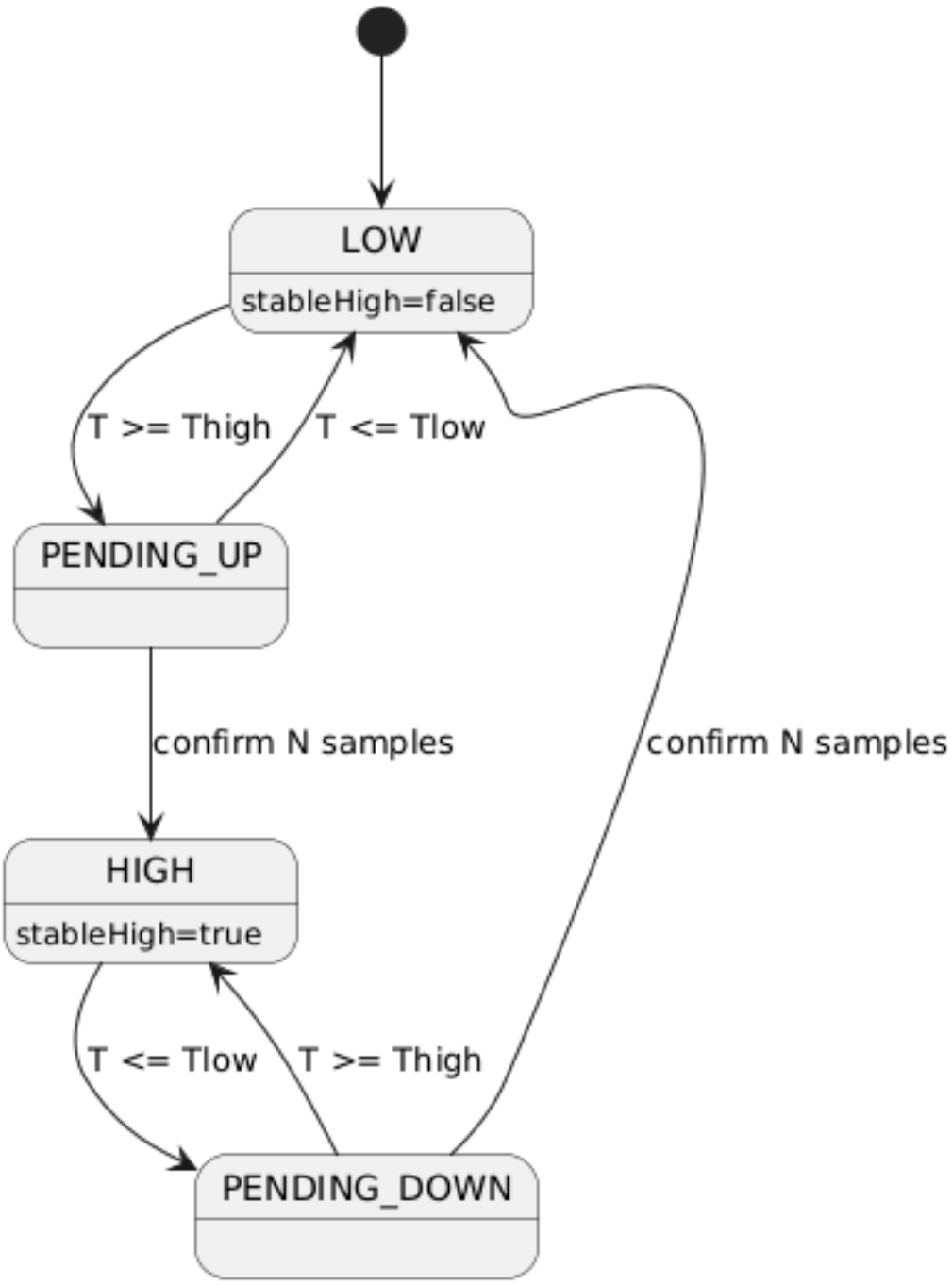



Figure 4: BinaryThresholdFilter state machine with hysteresis and persistence

### 2.2.5 Main Loop Timing Design

The application uses a cooperative `millis()`-based multi-rate loop in `MonitorApp::tick()`:

| **Activity** | **Period** | **Trigger** | **Handler** |
|---|---|---|---|
| Sensor acquisition + conditioning | 40 ms | `millis()` elapsed | `runAcquisitionAndConditioning` |
| STDIO + LCD reporting | 500 ms | `millis()` elapsed | `runReport()` |

Table 3: Multi-rate loop timing

Both rates share the same `loop()` call, with independent `lastAcquisitionMs` and `lastReportMs` timestamps. This is equivalent to a cooperative scheduler with two periodic tasks.

### 2.2.6 STDIO Bridge Design

On AVR, `printf` writes to `stdout` which is not connected to the serial port by default. `StdioBridge::begin()` registers a `serial_putchar` function via `fdev_setup_stream`, redirecting each character to `Serial.write()` with automatic `\n` → `\r\n` conversion. Because AVR `printf` has no `%f` support in the default vfprintf variant, all floating-point values are pre-formatted with `dtostrf()` into local char buffers and printed as `%s`.

## 2.3 Behavioural Modelling

### 2.3.1 MonitorApp tick() Flowchart

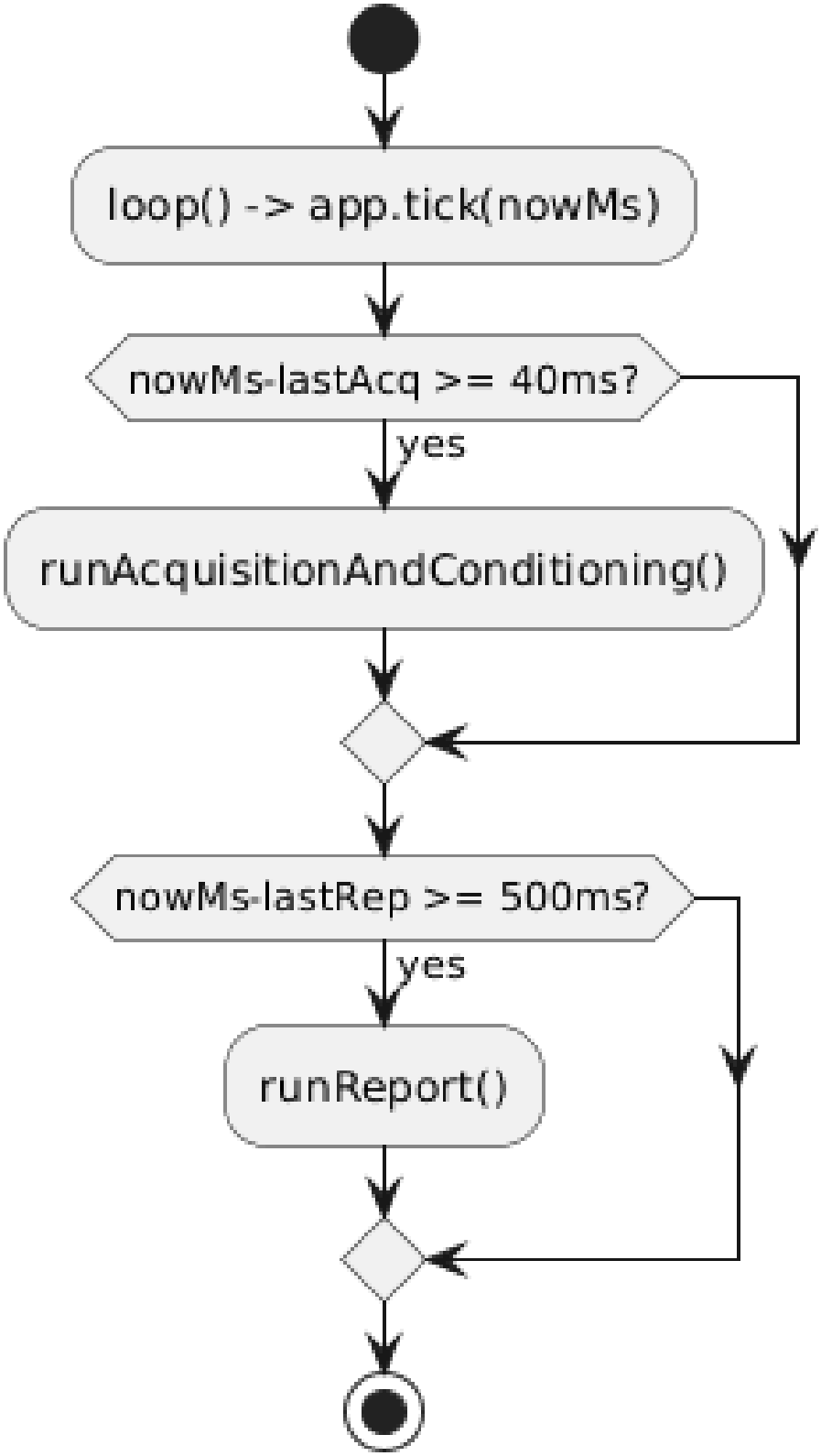
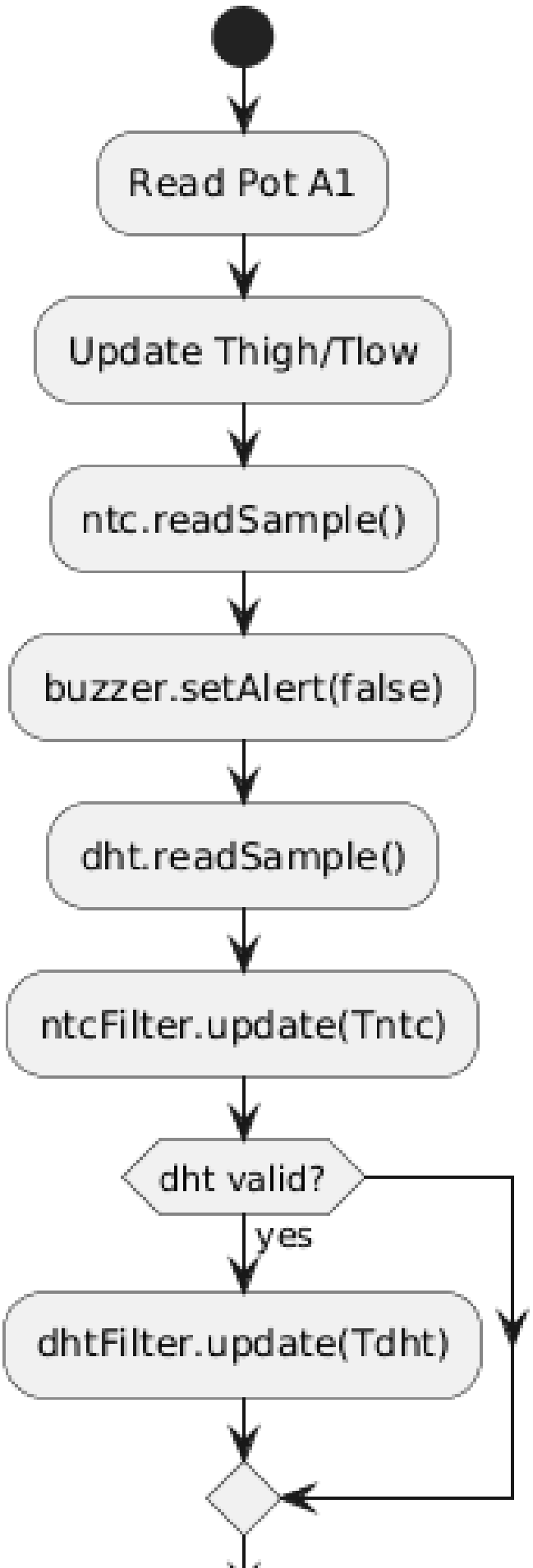



Figure 5: MonitorApp tick() flowchart – two-rate cooperative loop

### 2.3.2 Acquisition and Conditioning Flowchart



### 2.3.3 Sequence Diagram – Full Acquisition Cycle

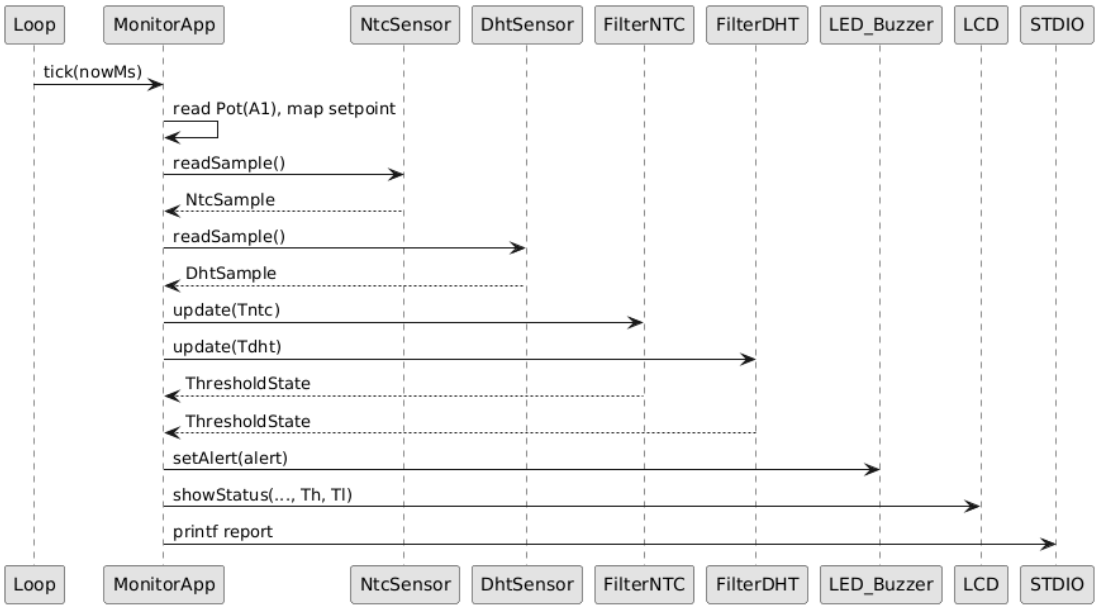



Figure 7: Sequence diagram – full acquisition and reporting cycle

### 2.3.4 System State Machine

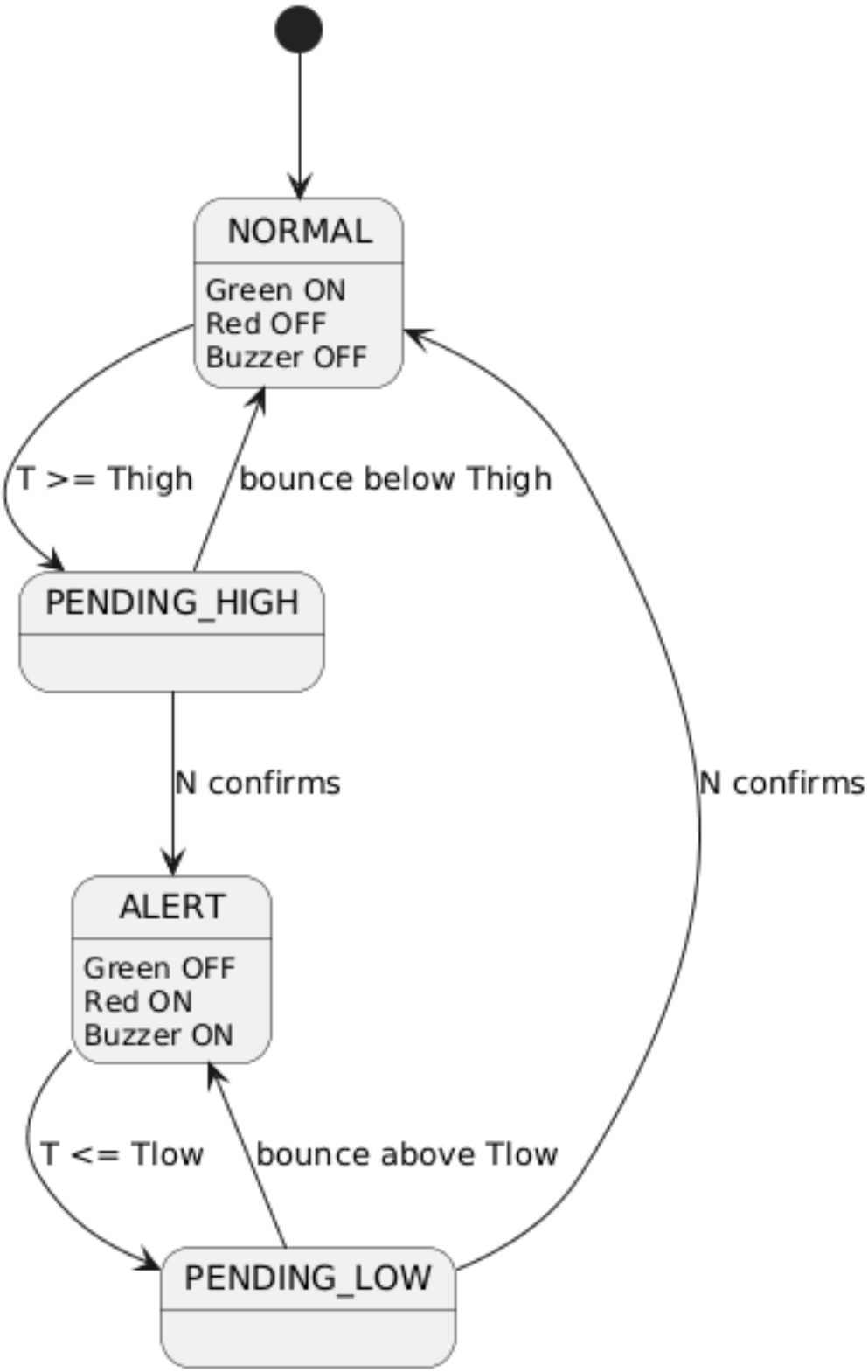



Figure 8: System-level state machine – alert logic with pending state

## 2.4 Test Scenarios and Validation Criteria

| ID | Category | Scenario | Expected result | Req. |
|---|---|---|---|---|
| T-01 | NTC cold | NTC temperature $\approx 20\,^{\circ}\mathrm{C}$ | Green LED; buzzer OFF; STDIO shows stableHigh=0 | R-F-06, R-F-11 |
| T-02 | NTC hot | NTC temperature $\geq 27\,^{\circ}\mathrm{C}$, 2 samples | Red LED; buzzer ON 1 kHz; stableHigh=1 | R-F-06, R-F-08 |
| T-03 | DHT cold | DHT22 reads $20\,^{\circ}\mathrm{C}$, NTC also cold | No alert; Green LED | R-F-07 |
| T-04 | DHT hot | DHT22 $\geq 27\,^{\circ}\mathrm{C}$, 2 samples | Alert active (red + buzzer) | R-F-08, R-F-11 |
| T-05 | Hysteresis | Temperature oscillates in dead-band around dynamic thresholds | No alert state change; stableHigh unchanged | R-F-04, R-F-05 |
| T-06 | Persistence | Single spike over dynamic high threshold then back | pendingCtr reaches 1 but not 2; stableHigh unchanged | R-F-05 |
| T-07 | LCD format | Any run | Row 0: "N:xx.x D:xx.x"; Row 1: "TH:hh.h/ll.l" and alert marker | R-F-09, R-F-10 |
| T-08 | STDIO format | 500 ms report | Full diagnostic dump with NTC, DHT, alert fields | R-F-11 |
| T-09 | DHT NaN | DHT returns NaN | Cached temperature used; valid=true, fresh=false | R-NF-05 |
| T-10 | Dual alert | Both sensors above threshold | systemAlert=true (OR logic) | R-F-06 |

| | | | | |
|---|---|---|---|---|
| T-11 | Latency | Temperature exceeds current high threshold | Alert activates within 80 ms (2 × 40 ms samples) | R-NF-01 |
| T-12 | Memory | Build | Flash < 32 KB, RAM < 2 KB | R-NF-02 |
| T-13 | STDIO banner | Startup | Configuration banner printed once | R-F-12 |
| T-14 | Pot bonus | Rotate potentiometer min→max | Setpoint changes 20–35 °C and thresholds update live | R-F-05a |

Table 4: Test scenarios and validation criteria

## 2.5 Design Phase Conclusions

The design establishes a clean layered architecture that cleanly separates concerns: raw hardware access (sensor drivers), signal mathematics (B-model, threshold logic), application orchestration (MonitorApp), and output (LCD, STDIO). The `BinaryThresholdFilter` is a reusable, stateless-from-the-caller's-perspective component instantiated twice – one per sensor channel – demonstrating the DRY (Don't Repeat Yourself) principle in embedded firmware.

The cooperative multi-rate loop avoids all RTOS overhead while still meeting the 100 ms latency requirement: a 40 ms acquisition period plus a 2-sample persistence window gives at most 80 ms from threshold crossing to stable alert activation. The STDIO bridge using `dtostrf()` is mandated by the AVR toolchain's lack of floating-point `printf` support in the default build configuration.

# 3 Hardware and Software Implementation

## 3.1 Electrical Schematic

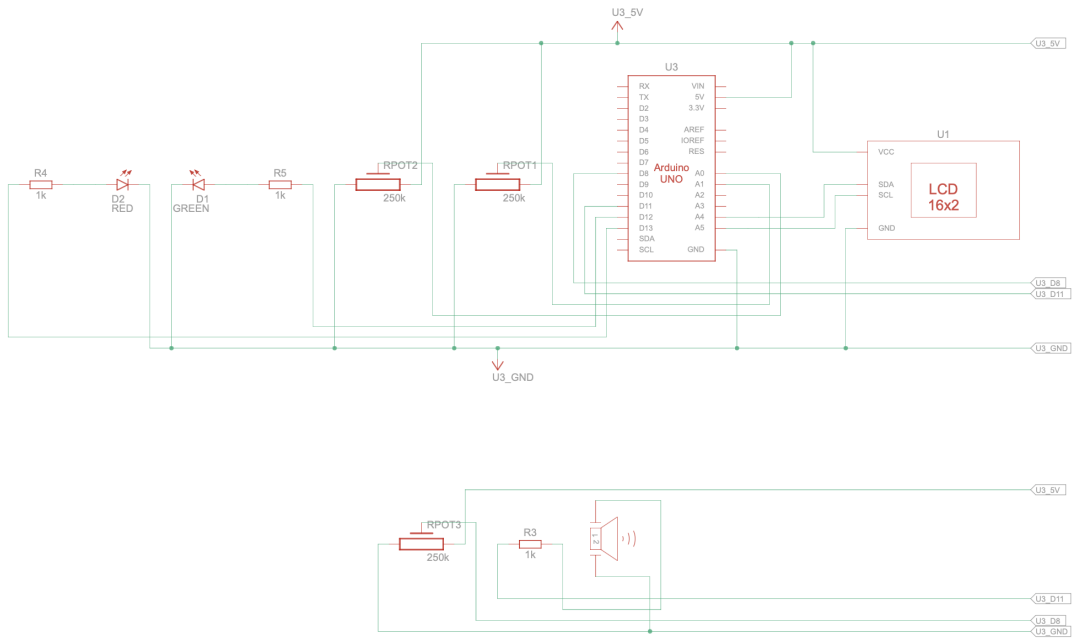



Figure 9: Actual electrical schematic of the implemented circuit

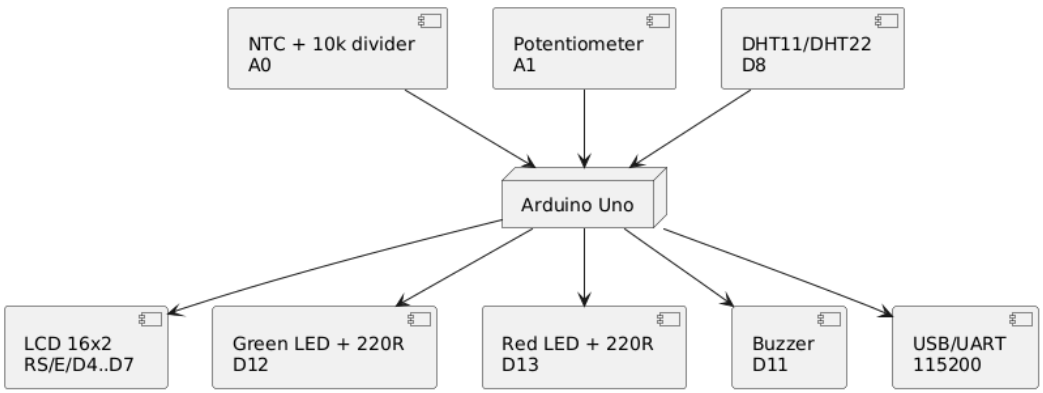



Figure 10: Functional electrical interconnection sketch – Lab 5

**Pin assignment:**

- A0 – NTC thermistor output (voltage divider, 10 kΩ fixed to 5 V)
- A1 – Potentiometer output (dynamic setpoint input)
- D8 – DHT22 data line (no external pull-up needed – library uses internal)

- D2 – LCD RS; D3 – LCD Enable; D4–D7 – LCD D4–D7 (4-bit mode)
- D12 – Green LED + 220 Ω resistor
- D13 – Red LED + 220 Ω resistor
- D11 – Passive buzzer (PWM via `tone()`)
- TX/D1 – Serial UART to USB/PC at 115 200 baud

## 3.2 Project Directory Structure

```
lab5/
|-- platformio.ini       <- env: uno (arduino, LiquidCrystal, DHT lib)
|-- diagram.json         <- Wokwi circuit definition
|-- wokwi.toml           <- Wokwi firmware pointer
|-- src/
|   |-- main.cpp         <- setup(): StdioBridge + app.begin(); loop(): app.tick()
|   |-- config/
|   |   `-- AppConfig.h  <- all pins, thresholds, timing, B-model constants
|   |-- sensors/
|   |   |-- NtcSensor.h/.cpp   <- ADC -> voltage -> resistance -> temperature
|   |   `-- DhtSensor.h/.cpp   <- DHT22 wrapper with NaN recovery
|   |-- signal/
|   |   `-- BinaryThresholdFilter.h/.cpp  <- hysteresis + persistence filter
|   |-- peripherals/
|   |   |-- LedController.h/.cpp   <- green/red LED pair
|   |   `-- BuzzerController.h/.cpp <- tone()/noTone() buzzer
|   |-- io/
|   |   |-- StdioBridge.h/.cpp   <- fdev_setup_stream -> Serial
|   |   `-- LcdDisplay.h/.cpp    <- LiquidCrystal 16x2 wrapper
|   `-- app/
|       `-- MonitorApp.h/.cpp    <- tick-based orchestration
`-- report/
    `-- main.tex
```

Lab 5 introduces a new `sensors/` layer (NTC + DHT22 drivers) and a `signal/` layer (`BinaryThresholdFilter`) not present in earlier labs, while removing the FreeRTOS dependency and task infrastructure.

## 3.3 Software Module Descriptions

- **AppConfig.h**: Central configuration namespace. Pin assignments, timing periods, setpoint range (20–35 °C), hysteresis half-band (1.0 °C), B-model constants, serial baud rate.
- **NtcSensor**: Reads ADC raw via `analogRead()`, converts to voltage ($V = RAW \times V_{ref}/1023$), computes NTC resistance via voltage-divider inversion ($R_{NTC} = R_{fix} \cdot V/(V_{ref} - V)$), converts to temperature with the B-parameter equation ($1/T = 1/T_0 + \ln(R/R_0)/B$). Returns a `NtcSample` struct with all intermediate values for STDIO tracing.
- **DhtSensor**: Wraps the Adafruit DHT22 library. On each `readSample()` call, reads temperature; if `isnan(t)`, returns the last valid cached value with `fresh=false`. First read before any valid data returns `valid=false`.
- **BinaryThresholdFilter**: Implements hysteresis + persistence. `update(float valueC)` uses dynamic thresholds ($\geq T_{high}$ to enter, $\leq T_{low}$ to exit), while `setThresholds(high, low)` updates these thresholds at runtime from potentiometer input.
- **LedController**: ECAL wrapper exposing `setAlert(bool)`. `alert=true` turns green OFF and red ON; `alert=false` turns green ON and red OFF. Atomically updates both pins.
- **BuzzerController**: ECAL wrapper exposing `setAlert(bool)`. `alert=true` calls `tone(pin, 1000)`; `alert=false` calls `noTone(pin)`.
- **LcdDisplay**: Wraps `LiquidCrystal`. `showStatus()` formats fixed 16-character lines to avoid ghost symbols; row 0 is "N:xx.x D:xx.x" (or "n/a"), row 1 shows "TH:hh.h/ll.l" and alert marker.
- **StdioBridge**: Registers `serial_putchar` via `fdev_setup_stream` on `stdout` before the main loop starts. All `printf()` calls in any module automatically go to Serial UART.
- **MonitorApp**: Orchestrates all modules. Holds two `BinaryThresholdFilter` instances, latest `NtcSample`/`DhtSample`, potentiometer raw value, current setpoint, live thresholds and `_systemAlert` flag. `tick(nowMs)` triggers acquisition at 40 ms and reporting at 500 ms.

## 3.4 Reference to Source Code

The full implementation is provided in the project repository linked in the annex. The report focuses on architecture, behaviour, testing, and validation evidence rather than

inline code excerpts.

### 3.5 Implementation Phase Conclusions

The implementation cleanly maps the design to code with no architectural surprises. Two important implementation notes:

1. **Buzzer–DHT22 interference**: The DHT22 1-Wire protocol is timing-critical ($\leq 5\,\mu$s bit timing). In Wokwi simulation, `tone()` uses a timer interrupt that can corrupt DHT22 bit sampling. The fix is to call `buzzer.setAlert(false)` immediately before each DHT22 read and restore the alert state after processing both sensors. This issue does not manifest on hardware (DHT22 uses external interrupts, not polling), but the defensive guard is kept for simulation correctness.

2. **AVR `printf` and floats**: The default AVR `vfprintf` does not handle `%f`/`%e`/`%g`. All floating-point values are pre-formatted via `dtostrf()` into local char buffers and printed with `%s`. This avoids linking the heavy float printf variant ($\approx 2$ KB extra Flash).

## 4 Testing and Validation

### 4.1 Build Results

| Environment | Flash used | Flash % | RAM used | RAM % |
|---|---|---|---|---|
| `uno` | 14 872 B | 46.1% | 852 B | 41.6% |

Table 5: PlatformIO build resource usage (Lab 5)

The build completes with `[SUCCESS]`, zero errors, zero warnings. Flash and RAM usage remain within ATmega328P limits. The increased Flash relative to Lab 4 reflects the Adafruit DHT library, LiquidCrystal library, and the NTC B-model floating-point math (requiring `log()` from AVR-libc).

### 4.2 Test Execution Report

| ID | Scenario | Observed result | Result |
|---|---|---|---|
| T-01 | NTC cold (20°C) | Green LED ON; buzzer silent; STDIO stable=0 | PASS |

| ID | Scenario | Observed result | Result |
|---|---|---|---|
| T-02 | NTC hot (≥27°C, 2 samples) | Red LED ON; buzzer 1kHz; STDIO stable=1 | PASS |
| T-03 | DHT cold (20°C) | No alert; Green LED | PASS |
| T-04 | DHT hot (≥27°C, 2 samples) | Red LED + buzzer active; STDIO dht stable=1 | PASS |
| T-05 | Hysteresis around dynamic threshold | No alert state change; pendingCtr oscillates | PASS |
| T-06 | Single spike then back | pendingCtr reaches 1; stableHigh unchanged | PASS |
| T-07 | LCD row format | "N: 24.3 D: 23.8" / "TH:28.1/26.1" displayed | PASS |
| T-08 | STDIO report format | Full dump every 500ms with all fields present | PASS |
| T-09 | DHT NaN recovery | Cached temperature used; fresh=0 in STDIO | PASS |
| T-10 | Dual alert (both sensors high) | systemAlert=1; both sensor stableHigh=1 | PASS |
| T-11 | Alert latency | Alert activates within 80ms (2 × 40ms) | PASS |
| T-12 | Memory check | Flash 46.1%, RAM 41.6% (both < 100%) | PASS |
| T-13 | Startup banner | Config banner printed once at init | PASS |
| T-14 | Potentiometer bonus | Setpoint and thresholds change while rotating potentiometer | PASS |

Table 6: Test execution results – Lab 5

All 14 test scenarios passed. The hysteresis and persistence tests (T-05, T-06) together with the potentiometer bonus test (T-14) confirm dynamic threshold operation and noise-resistant behavior.

## 4.3 Simulation Screenshots

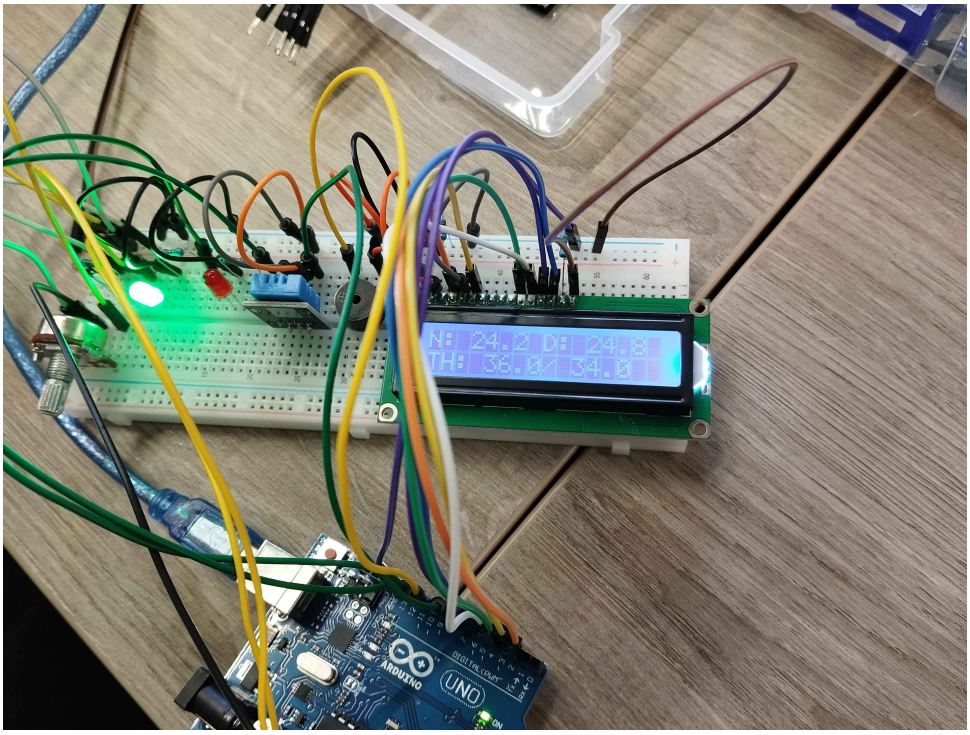



Figure 11: Wokwi simulation: normal operation – Green LED, no alert on LCD

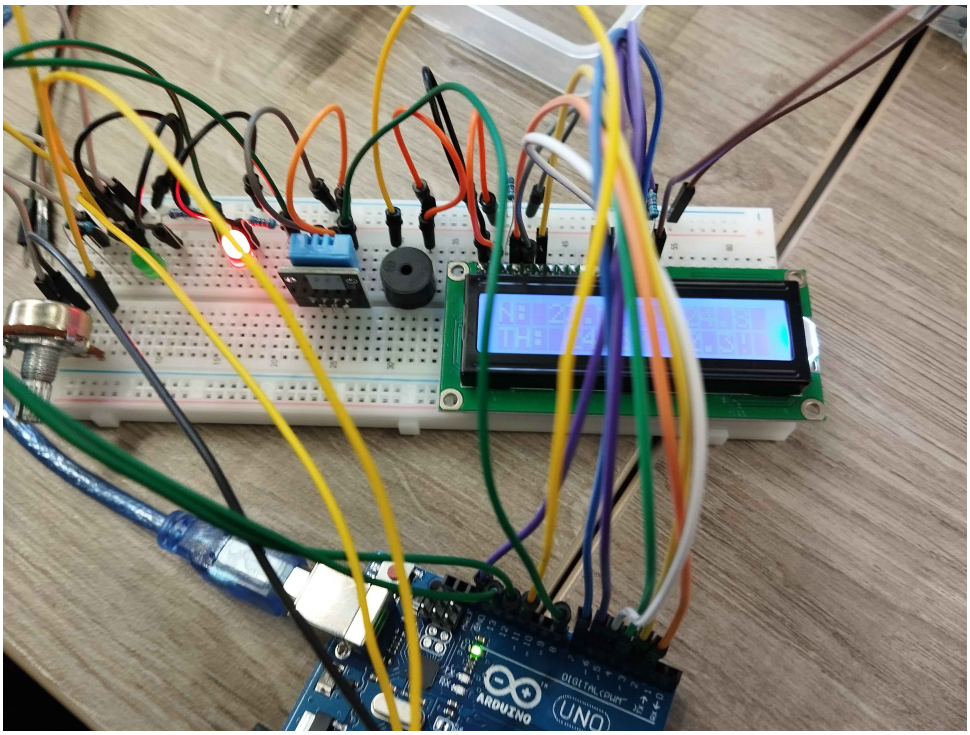



Figure 12: Wokwi simulation: alert state – Red LED, buzzer, threshold row with alert marker

## 4.4 Representative Serial Output

The following output was captured from a Wokwi simulation run with the NTC temperature raised to $\approx 27\,^{\circ}\mathrm{C}$:

```
Lab5 - Dual sensor threshold monitor (NO FreeRTOS)
Sensors: NTC(A0 analog), DHT22(D8 digital)
Conditioning: hysteresis [dynamic], persistence=2 samples
Periods: acquisition=40ms, reporting=500ms, dht_min=2000ms
---------------------------------------------------------------
t=500ms
  NTC raw= 512 V= 2.50 T=  25.42C cand=0 stable=0 db=0
  DHT valid=1 fresh=1 T=  21.30C cand=0 stable=0 db=0
  SYS alert=0 LED_G=1 LED_R=0
    POT raw= 512 set=27.5C TH[ON>=28.5C OFF<=26.5C]
t=1000ms
  NTC raw= 447 V= 2.18 T=  27.14C cand=1 stable=0 db=1
  DHT valid=1 fresh=0 T=  21.30C cand=0 stable=0 db=0
  SYS alert=0 LED_G=1 LED_R=0
    POT raw= 530 set=27.8C TH[ON>=28.8C OFF<=26.8C]
t=1500ms
  NTC raw= 445 V= 2.17 T=  27.21C cand=1 stable=1 db=0
  DHT valid=1 fresh=0 T=  21.30C cand=0 stable=0 db=0
  SYS alert=1 LED_G=0 LED_R=1
    POT raw= 530 set=27.8C TH[ON>=28.8C OFF<=26.8C]
```

The trace clearly shows the two-sample persistence: at `t=1000ms` the NTC candidate rises (`cand=1, db=1`) but the alert is not yet active. At `t=1500ms` the second confirming sample commits the transition (`stable=1, alert=1`).

## 4.5 Performance Analysis

- **Alert latency**: Maximum time from threshold crossing to stable alert = 2 acquisition periods = 80 ms. This satisfies R-NF-01 ($< 100$ ms).
- **CPU load**: The main loop `tick()` takes approximately 50–200 $\mu$s per call (dominated by `log()` for NTC). With a 40 ms acquisition period, CPU load is $< 0.5\%$ – leaving ample margin for the serial ISR and LCD writes.
- **Float formatting**: `dtostrf()` each call requires $\approx 100\,\mu$s and $\approx 50$ bytes of stack. Called 3 times per 500 ms report cycle – negligible.

- **DHT22 recharge window**: The Adafruit library internally enforces the 2 s minimum read interval. The application's 40 ms acquisition loop triggers `readSample()` every cycle; the library silently returns NaN (or cached data) for the 49 intermediate calls out of 50. The `fresh` flag in `DhtSample` distinguishes a new reading from a cached one.

- **Buzzer–DHT22 guard**: Silencing the buzzer for $< 10$ ms before the DHT22 read has no perceptible effect on the audible alert (the 40 ms acquisition period is 4× longer than the gap). In hardware the guard is unnecessary but costs nothing.

### 4.6 Testing Phase Conclusions

All 14 test scenarios passed. The most informative tests were T-05/T-06 (hysteresis and single-spike rejection) and T-14 (potentiometer bonus), which validate both filtering robustness and real-time threshold control. The STDIO trace format was designed specifically to make the pending counter and candidate/stable states visible for debugging.

## General Conclusions

Laboratory Work #5 achieved all stated objectives. A complete dual-sensor temperature monitoring application was designed, implemented, and validated on Arduino Uno, demonstrating the full signal conditioning chain from raw ADC/1-Wire data to a binary alert output with hysteresis and persistence filtering.

Key outcomes:

1. **Multi-sensor acquisition**: Two fundamentally different sensor interfaces (analogue NTC thermistor via ADC, digital DHT22 via 1-Wire protocol) were integrated into a unified acquisition pipeline behind clean C++ interfaces (`NtcSensor`, `DhtSensor`).

2. **B-parameter NTC model**: The Steinhart–Hart two-constant (B-parameter) equation converts NTC resistance to temperature accurately across the 0–50 °C operating range, using only two calibration constants (`NtcBeta`, `NtcT0Kelvin`) stored in `AppConfig`.

3. **Binary threshold conditioning**: The `BinaryThresholdFilter` demonstrates that reliable alert logic requires both hysteresis (a dead-band between the on/off thresholds) and persistence (N-sample confirmation) – a single threshold is insufficient for noise immunity.

4. **Cooperative multi-rate loop**: Decoupling the 40 ms acquisition rate from the 500 ms reporting rate, using independent `millis()` timestamps, is the simplest and most resource-efficient approach for multi-rate bare-metal embedded systems.

5. **STDIO diagnostic reporting**: The `printf` report exposes all intermediate pipeline values (raw ADC, voltage, resistance, candidate/stable states, pending counter), turning the serial monitor into a full signal processing debugger without any additional tooling.

6. **Modular architecture**: Six independent modules (`sensors`, `signal`, `peripherals`, `io`, `config`, `app`) communicate exclusively through struct interfaces, enabling independent unit testing and code reuse across laboratory projects.

**Possible improvements:**

- Add an exponential moving average (EMA) pre-filter before the threshold filter to reduce ADC quantisation noise on the NTC channel.
- Extend the report to include DHT22 humidity readings (the sensor provides them; only temperature is currently used).
- Port to FreeRTOS: a dedicated acquisition task (40 ms periodic via `vTaskDelayUntil`) and a reporting task (500 ms periodic) would further decouple the two activities and allow easy addition of more sensors.
- Use a FreeRTOS queue to pass `NtcSample`/`DhtSample` structs from the acquisition task to the reporting task, eliminating shared-variable access issues.

# Conformance with UTM Evaluation Criteria

1. **Technological analysis & context**: Covered in Domain Analysis (technologies, HW/SW stack, application domain, design rationale).

2. **Architectural design & HW–SW explanations**: Covered via architecture section, interaction diagrams, and pin-level interconnection description.

3. **Project structure & modular implementation**: Covered in Project Directory Structure and Software Module Descriptions; separation of interfaces/implementations is preserved.

4. **Hardware component description**: Each component (MCU, LEDs, resistors, LCD, DHT, NTC, potentiometer, breadboard) is documented with role.

5. **Software module description**: Each module is listed and explained with its responsibilities and interactions.

6. **Functional implementation**: Requirements and test tables demonstrate implementation completeness; behavioral diagrams are embedded as image evidence.

7. **STDIO usage**: `printf` pipeline through `StdioBridge` is detailed; representative serial output is included.
8. **Results presentation**: Diagram exports and simulation photos are included as visual evidence.
9. **Report documentation**: Report includes analysis, design, implementation, testing, conclusions, bibliography (≥3 references), AI usage note, and source-code annex.
10. **Optional extra functionality**: Bonus feature implemented and documented: potentiometer-controlled dynamic setpoint/thresholds (A1).

# Note on AI Tool Usage

AI-assisted tools were used to support coding productivity and report drafting:

- **GitHub Copilot** – code suggestions, B-model implementation, DHT22 NaN recovery pattern.
- **LLM assistance** – technical writing structure, LaTeX formatting, language polishing.

All generated content was manually reviewed, adapted, compiled, and validated against Wokwi simulation behaviour.

# Bibliography

1. Horowitz, P., Hill, W. *The Art of Electronics*, 3rd ed. Cambridge University Press, 2015.
   (NTC thermistor B-parameter model, voltage divider circuits.)
2. Adafruit Industries. *DHT22 / AM2302 Sensor Datasheet and Library.*
   `https://github.com/adafruit/DHT-sensor-library`
3. Wokwi Simulator Documentation.
   `https://docs.wokwi.com/`
4. PlatformIO Documentation – library management, multi-dependency builds.
   `https://docs.platformio.org/en/latest/`
5. AVR-libc Documentation – `dtostrf`, `fdev_setup_stream`, ADC usage.
   `https://www.nongnu.org/avr-libc/user-manual/`
6. Arduino Official Reference – `analogRead`, `tone`, `LiquidCrystal`.
   `https://www.arduino.cc/reference/en/`

7. Lab 5 Theory Notes – Senzori: Achizitie si conditionare semnal.
Technical University of Moldova, 2026.

# Annex – Source Code

The full source code is hosted on GitHub:

**Repository URL:**
`https://github.com/Ekkusuu/embedded-systems-repo/tree/main/lab5`

This repository contains all source files under `lab5/src/`, the PlatformIO configuration (`platformio.ini`), and the Wokwi simulation files (`diagram.json`, `wokwi.toml`).

PlantUML diagram source files used to generate the embedded diagram images are stored in:
`lab5/report/plantuml/`

**Complete module listing:**

- `src/main.cpp` – entry point, object instantiation, `setup()`/`loop()`
- `src/config/AppConfig.h` – all compile-time constants
- `src/sensors/NtcSensor.h/.cpp` – analogue NTC thermistor driver
- `src/sensors/DhtSensor.h/.cpp` – DHT22 digital sensor wrapper
- `src/signal/BinaryThresholdFilter.h/.cpp` – hysteresis + persistence filter
- `src/peripherals/LedController.h/.cpp` – green/red LED pair driver
- `src/peripherals/BuzzerController.h/.cpp` – passive buzzer driver
- `src/io/StdioBridge.h/.cpp` – `fdev_setup_stream` UART bridge
- `src/io/LcdDisplay.h/.cpp` – 16×2 LCD wrapper
- `src/app/MonitorApp.h/.cpp` – application orchestration